

UNIVERSITY OF EDINBURGH

INTERNSHIP REPORT

M1 INFORMATIQUE CONCEPTS ET APPLICATIONS

Varieties of Semiring Solvers via Multicategorical and Algebraic Constructions

Intern:

Amaury MAZOYER
ENS de Lyon,
France

Supervisor :

Ohad KAMMAR
University of Edinburgh,
United Kingdom

May 4 - July 31 2026



THE UNIVERSITY *of* EDINBURGH
informatics

Contents

1	Introduction	2
2	Preliminaries	2
2.1	Syntax	2
2.2	Free models/algebras	3
2.3	Free extensions	4
2.4	How the fral/frex solving works	4
2.5	The distributive combination of theories	6
2.6	Monads and distributive laws	6
3	Categorical approach to the construction of distributive combinations	7
3.1	Challenge about constructing distributive combinations	8
3.2	Initial result about the free algebra for the distributive combination	8
3.3	Failure of the first result	8
3.4	Corrected result	9
3.5	An attempt at constructing the free extension for the distributive combination	10
4	Implementation	10
4.1	Setoids	10
4.2	Implementing the distributive combination	10
4.3	Performance evaluation	11
5	Comparison with existing solvers	11
5.1	Reflection	11
5.2	Almost-rings	12
5.3	Soundness and completeness	13
5.4	Comparison	13
5.4.1	Supported theories	13
5.4.2	Additional equalities	15
5.4.3	Other FREX capabilities	15
5.4.4	Runtime comparison	16
6	Conclusion	16
6.1	Related work	16
6.2	Future work	16
	Appendices	18
A	Internship context	18
B	Source code	18
C	Proof of Theorem 5	18
D	Research notes	18

1 Introduction

- motivation → algebraic simplification is a pain to do manually in dependently typed languages/in proof assistants
- solvers in other languages (Rocq [Grégoire & Mahboubi, 2005], Agda [Kidney, 2019])
- state of the art: frex debut [Yallop et al., 2018], idris frex [Allais et al., 2025] and involution construction [Allais et al., 2023]
- presentation of the contributions:
 - finishing Ohad’s construction proof and correcting the needed hypothesis
 - proof of freeness of the construction
 - formalising the distributive combination theory in Idris
 - implementing the distributive combination semantics
 - implementing some free algebras
 - testing and comparison

2 Preliminaries

FREX is based on some concepts of *universal algebra*, which gives generic descriptions of algebraic structures and relations between them [Burris & Sankappanavar, 1981, Chp. II]. We’ll summarise the relevant concepts, which have been formalised in Idris 2 [Allais et al., 2025]. We’ll use the example of monoids to illustrate these concepts along the way. We’ll also explain how the solving works, and we’ll define the distributive combination of theories formally.

2.1 Syntax

A *finitary signature* $\Sigma = (\text{op } \Sigma, \text{arity})$ consists of a set $\text{op } \Sigma$ of *operators*, and an assignment $\text{arity} : \text{op } \Sigma \rightarrow \mathbb{N}$ associating to each operator in $\text{op } \Sigma$ its *arity*. The usual signature Σ_{Mon} of monoids consists of two operators $(\cdot)$ and 1 , with respective arities 2 and 0. We’ll write $\Sigma_{\text{Mon}} := \{(\cdot) : 2, 1 : 0\}$.

An *algebra* $A = (\underline{A}, A \llbracket - \rrbracket)$ over a signature Σ consists of a set $\underline{A}$ called the *carrier* of A , and a function $A \llbracket f \rrbracket : \underline{A}^n \rightarrow \underline{A}$ called the *interpretation* of f in A for each operator $f : n \in \Sigma$. This gives the semantics to the algebraic language determined by the signature. There may be different algebras for a given signature and carrier set. One can equip the set $\mathbb{N}$ of natural numbers with the monoid structure $(\mathbb{N}, (+), 0)$, $(\mathbb{N}, (\cdot), 0)$ or even $(\mathbb{N}, \text{max}, 0)$.

Given a set X of variables and a signature Σ , the set of Σ -terms over X , denoted $\text{Term}_\Sigma(X)$, is defined by induction:

$$\frac{x \in X}{\text{var } x \in \text{Term}_\Sigma(X)} \quad \frac{t_i \in \text{Term}_\Sigma(X) \quad f : n \in \Sigma}{f(t_1, \dots, t_n) \in \text{Term}_\Sigma(X)}$$

We’ll usually write $x \in \text{Term}_\Sigma(X)$ instead of $\text{var } x \in \text{Term}_\Sigma(X)$. We define *equations in context* $X \vdash l = r$ as triples (X, l, r) where X is a set of variables and l, r are terms in context X called the *left-hand-side* (LHS) and *right-hand-side* (RHS). The associativity equation in Σ_{Mon} is then defined as $x, y, z \vdash x \cdot (y \cdot z) = (x \cdot y) \cdot z$.

An *environment* ρ for a context X , or X -environment, in an algebra A is a function $\rho : X \rightarrow \underline{A}$ that assigns to each variable in X an element of the carrier of A . Given

a term $t \in \text{Term}_\Sigma(X)$, denoted $X \vdash t$, the algebra A gives an *interpretation* function $A \llbracket t \rrbracket : \underline{A}^X \rightarrow \underline{A}$. For an X -environment ρ , $A \llbracket t \rrbracket \rho$ is defined by induction:

$$A \llbracket \text{var } x \rrbracket \rho := \rho x \quad A \llbracket \text{op}(t_1, \dots, t_n) \rrbracket \rho := A \llbracket \text{op} \rrbracket (A \llbracket t_1 \rrbracket \rho, \dots, A \llbracket t_n \rrbracket \rho)$$

For example, in the Σ_{Mon} -algebra $A = (\mathbb{N}, (+), 0)$, one can interpret the term $x \cdot (y \cdot 1)$ in the environment $\{x \mapsto 1, y \mapsto 2\}$, yielding $A \llbracket x \cdot (y \cdot 1) \rrbracket = 1 + (2 + 0) = 3$

An equation $X \vdash l = r$ between terms in $\text{Term}_\Sigma(X)$ is *valid* in an algebra A if for every environment $\rho : X \rightarrow \underline{A}$, we have $A \llbracket l \rrbracket \rho = A \llbracket r \rrbracket \rho$. We write this $A \models (X \vdash l = r)$. For example, the Σ_{Mon} algebras presented so far validate the associativity axiom, but interpreting the binary operation as subtraction over the integers $(\mathbb{Z}, (-), 0)$ does not: taking the environment $\{x \mapsto 0, y \mapsto 0, z \mapsto 1\}$, we have $0 - (0 - 1) = 1 \neq -1 = (0 - 0) - 1$.

A *presentation* (or *theory*) $\mathcal{T} = \langle \Sigma_{\mathcal{T}}, \text{Ax}_{\mathcal{T}} \rangle$ consists of a signature $\Sigma_{\mathcal{T}}$ and a set $\text{Ax}_{\mathcal{T}}$ of Σ -equations in context. A $\mathcal{T}$ -*model* A is a $\Sigma_{\mathcal{T}}$ algebra validating all of the axioms in $\text{Ax}_{\mathcal{T}}$. The **Monoid** presentation consists of the associativity axiom paired with the neutrality axioms $x \vdash x \cdot 1 = x$ and $x \vdash 1 \cdot x = x$ over the signature Σ_{Mon} .

Let $A = (\underline{A}, A \llbracket - \rrbracket)$ and $B = (\underline{B}, B \llbracket - \rrbracket)$ be algebras over the same signature Σ . A *homomorphism* of Σ -algebras from A to B is a function $h : \underline{A} \rightarrow \underline{B}$ such that for every operator $f : n \in \Sigma$ and every $a_1, \dots, a_n \in \underline{A}$ we have:

$$B \llbracket f \rrbracket (h(a_1), \dots, h(a_n)) = h(A \llbracket f \rrbracket (a_1, \dots, a_n))$$

In other words, a homomorphism is a semantics-preserving function between the carriers of the algebras. For example, complex conjugation is an homomorphism between the Σ_{Mon} -algebras over the complex numbers, since $\overline{z_1 + z_2} = \overline{z_1} + \overline{z_2}$, $\overline{0} = 0$ and $\overline{z_1 \cdot z_2} = \overline{z_1} \cdot \overline{z_2}$, $\overline{1} = 1$. A *homomorphism* of $\mathcal{T}$ -models is a homomorphism of $\Sigma_{\mathcal{T}}$ -algebras between the underlying algebras of the models.

2.2 Free models/algebras

Let $\mathcal{T}$ be a presentation and X a set of variables. A $\mathcal{T}$ -model over X is just a pair $\langle A, e \rangle$ where A is a $\mathcal{T}$ -model and $e : X \rightarrow \underline{A}$ is an X -environment in this algebra. This concept is used to formalise the fral solving process in §2.4.

A *morphism* $h : \langle A, \rho_A \rangle \rightarrow \langle B, \rho_B \rangle$ between $\mathcal{T}$ -models over X is a $\mathcal{T}$ -model homomorphism that makes the diagram on the right commute. A *free $\mathcal{T}$ -model over X* is a $\mathcal{T}$ -model $\langle F(X), \text{env} \rangle$ such that for every $\mathcal{T}$ -model A over X , there exists a unique morphism of $\mathcal{T}$ -models from $\langle F(X), \text{env} \rangle$ to A . We will often use the term "free algebra", abbreviated "fral", when $\mathcal{T}$ is implicit. The existence-and-uniqueness property is called the *universal property* of the free algebra.

$$\begin{array}{ccc} & & \underline{A} \\ & \nearrow^{\rho_A} & \downarrow h \\ X & = & \\ & \searrow_{\rho_B} & \underline{B} \end{array}$$

Thanks to the universal property, all free $\mathcal{T}$ -models over X are isomorphic and therefore we often talk about *the* free algebra over X . Informally, the free algebra for a theory is the simplest and most general model validating the theory, in the sense that the only equations validated by the free algebra are the ones from the theory and the ones that we can deduce from them. This is ensured by the universal property. The free commutative monoid over the set of variables $\{x_1, x_2, x_3\}$ can be represented by $\mathbb{N}^3$. Addition is coordinate wise, the 0 element is $(0, 0, 0)$ and env is the function that sends x_1 to $(1, 0, 0)$, x_2 to $(0, 1, 0)$ and x_3 to $(0, 0, 1)$. The unique morphism out of this algebra into any commutative monoid A is the function sending (a_1, a_2, a_3) to $a_1(\rho_A(x_1)) + a_2(\rho_A(x_2)) + a_3(\rho_A(x_3))$.

Every presentation $\mathcal{T}$ induces an equivalence relation between terms using the deduction rules of equational logic. Explicitely, we define a *provability* relation $X \vdash l = r$ inductively as in Figure 1.

$$\begin{array}{c}
\frac{\text{ax} \in \text{Ax}_{\mathcal{T}}}{\text{ax}} \quad (\text{axiom}) \quad X \vdash t = t \quad (\text{reflexivity}) \quad \frac{X \vdash l = r}{X \vdash r = l} \quad (\text{symmetry}) \\
\frac{X \vdash l = m \quad X \vdash m = r}{X \vdash l = r} \quad (\text{transitivity}) \quad \frac{X \vdash l = r \quad \theta : X \rightarrow \text{Term}_{\Sigma_{\mathcal{T}}} Y}{Y \vdash l[\theta] = r[\theta]} \quad (\text{substitution}) \\
\frac{\text{for } i = 1, \dots, n : X \vdash l_i = r_i}{X \vdash \text{op}(l_1, \dots, l_n) = \text{op}(r_1, \dots, r_n)} \quad (\text{congruence})
\end{array}$$

Figure 1: Provability

Allais et al. [2025] proves that there is an isomorphism of $\mathcal{T}$ -models over X

$$\langle F(X), \text{env} \rangle \cong \langle (\text{Term}_{\Sigma_{\mathcal{T}}} X) / (X \vdash_{\mathcal{T}}) =: Q, \text{env}' \rangle$$

with $\text{env}' : \begin{cases} X \rightarrow Q \\ x \mapsto [\text{var } x] \end{cases}$. Therefore free algebras represent terms modulo semantics.

2.3 Free extensions

An *extension of A by X* is a triple $\langle B, h, e \rangle$ where B is a $\mathcal{T}$ -algebra, $h : A \rightarrow B$ is a $\mathcal{T}$ -homomorphism and $e : X \rightarrow B$ is a function.

Let $b_1 = \langle B_1, h_1, e_1 \rangle, b_2 = \langle B_2, h_2, e_2 \rangle$ be two extensions of A by X . A *morphism of extensions* $h : b_1 \rightarrow b_2$ is a $\mathcal{T}$ -homomorphism $h : B_1 \rightarrow B_2$ that makes the diagram on the right commute. An extension $\langle A[X], \text{sta}, \text{dyn} \rangle$ is *free* when, for every extension $\langle B, h, e \rangle$, there is a unique extension morphism $[B, h, e] : \langle A[X], \text{sta}, \text{dyn} \rangle \rightarrow \langle B, h, e \rangle$. Free extension is abbreviated "frex", and a free extension of A by X is usually denoted $A[X]$. Just like for free algebras, all free extensions of A by X are isomorphic to each other. Let A be a commutative monoid. Its frex by a finite set $X = \{x_1, \dots, x_n\}$ of variables can be represented by $A[X] := \underline{A} \times \mathbb{N}^n$. The remaining structure is defined by:

$$\begin{array}{ccccc}
& & B_1 & & \\
e_1 \nearrow & & \downarrow h & \nwarrow h_1 & \\
X & & & & A \\
e_2 \searrow & & B_2 & \swarrow h_2 &
\end{array}$$

$$\begin{aligned}
\langle a, \langle k_{11}, \dots, k_{1n} \rangle \rangle + \langle b, \langle k_{21}, \dots, k_{2n} \rangle \rangle &:= \langle a + b, \langle k_{11} + k_{21}, \dots, k_{1n} + k_{21} \rangle \rangle \\
0 &:= \langle 0, \langle 0, \dots, 0 \rangle \rangle \quad \text{sta } a := \langle a, \langle 0, \dots, 0 \rangle \rangle \quad \text{dyn } x_i := \left\langle 0, \left\langle 0, \dots, 0, \underset{i}{1}, 0, \dots, 0 \right\rangle \right\rangle \\
[B, h, e] \langle a, \langle k_1, \dots, k_n \rangle \rangle &:= h a + k_1 \cdot e x_1 + \dots + k_n \cdot e x_n
\end{aligned}$$

Similarly to §2.2, there is a canonical isomorphism between any free extension and the quotient frex $\langle (\text{Term}_{\Sigma_{\mathcal{T}_A}} X) / (X \vdash_{\mathcal{T}_A}) =: Q, \text{sta}', \text{dyn}' \rangle$ with $\text{sta}' : \begin{cases} A \rightarrow Q \\ a \mapsto [\underline{a}] \end{cases}$ and $\text{dyn}' : \begin{cases} X \rightarrow Q \\ x \mapsto [\text{var } x] \end{cases}$. $\mathcal{T}_A$ is the presentation where the operators are the ones of $\mathcal{T}$ together with newly adjoined constants $\underline{a}$ for every $a \in \underline{A}$, and whose axioms are the combination of the axioms in $\mathcal{T}$ and the evaluation axioms $\vdash \text{op}(\underline{a}_1, \dots, \underline{a}_n) = A \llbracket \text{op} \rrbracket (a_1, \dots, a_n)$ for every $\text{op} : n \in \Sigma_{\mathcal{T}}$ and $(a_1, \dots, a_n) \in \underline{A}^n$ [Allais et al., 2023].

2.4 How the fral/frex solving works

We'll explain how the fral solving works, and then explain how to adapt this for free extensions. This has been formalised in Idris 2 [Allais et al., 2025], see §4.

Suppose that we want to prove that the equation $x + (y + x) + y = x + x + y + y$ holds in a commutative monoid A . We evaluate the LHS and RHS of the equation in the free algebra,

both give the representative $(2, 2)$. We then prove the equality in the free algebra, this is simple reflexivity here, and we use the universal property of the free algebra to conclude of the validity of the equation in A .

In the general setting, let X be a set of variables, $\mathcal{T}$ be a theory, $\langle A, \text{env}_A \rangle$ be a $\mathcal{T}$ -model over X and $\langle FX, \text{env}_{FX} \rangle$ be a free $\mathcal{T}$ -model over X . We want to prove that $A \llbracket l \rrbracket \text{env}_A = A \llbracket r \rrbracket \text{env}_A$ for some equation $X \vdash l = r$. To do this, we need a proof that $FX \llbracket l \rrbracket \text{env}_{FX} = FX \llbracket r \rrbracket \text{env}_{FX}$. Suppose that we have this proof. The universal property of FX gives an homomorphism $h : \underline{FX} \rightarrow \underline{A}$ such that $h \circ \text{env}_{FX} = \text{env}_A$.

Lemma 1 (Homomorphisms preserve term semantics)

Let A, B be two $\mathcal{T}$ -algebras, $X \vdash t$ be a term and $h : A \rightarrow B$ be a $\mathcal{T}$ -homomorphism. Then for all function $\text{env} : X \rightarrow \underline{A}$, $h(A \llbracket t \rrbracket \text{env}) = B \llbracket t \rrbracket (h \circ \text{env})$.

Proof

This is a simple induction, since h preserves the semantics of the operators in $\mathcal{T}$. $\square$

Lemma 2 (Homomorphisms preserve equations)

Let A, B be two $\mathcal{T}$ -algebras, $\text{env} : X \rightarrow \underline{A}$ be a function, $X \vdash l = r$ be an equation and $h : A \rightarrow B$ be a $\mathcal{T}$ -homomorphism. Suppose that $A \llbracket l \rrbracket \text{env} = A \llbracket r \rrbracket \text{env}$. Then $B \llbracket l \rrbracket (h \circ \text{env}) = B \llbracket r \rrbracket (h \circ \text{env})$.

Proof

$$\begin{aligned} B \llbracket l \rrbracket (h \circ \text{env}) &= h(A \llbracket l \rrbracket \text{env}) && \text{since } h \text{ preserves term semantics} \\ &= h(A \llbracket r \rrbracket \text{env}) && \text{since } A \llbracket l \rrbracket \text{env} = A \llbracket r \rrbracket \text{env by hypothesis} \\ &= B \llbracket r \rrbracket (h \circ \text{env}) && \text{since } h \text{ preserves term semantics} \end{aligned}$$

$\square$

Now we can finally prove our equality in A . Since $FX \llbracket l \rrbracket \text{env}_{FX} = FX \llbracket r \rrbracket \text{env}_{FX}$, h is a homomorphism and homomorphisms preserve equations, $A \llbracket l \rrbracket (h \circ \text{env}_{FX}) = A \llbracket r \rrbracket (h \circ \text{env}_{FX})$. But $h \circ \text{env}_{FX} = \text{env}_A$, therefore $A \llbracket l \rrbracket \text{env}_A = A \llbracket r \rrbracket \text{env}_A$.

Proving equalities in arbitrary models with free algebras then just comes down to proving equalities in the free algebra. Since free algebras are canonically isomorphic to the quotient algebra of terms quotiented by the provability relation, if an equality is indeed provable, then the two associated terms in the free algebra should also be equal. The difficulty only resides in the definition of equality in the free algebra.

Proving equalities between partially static terms using free extensions is not much harder. Instead of considering an equation on X , we consider an equation on $X \sqcup \underline{A}$. For an extension $\langle A, \text{sta}, \text{dyn} \rangle$, the environment we use is defined as follows:

$$\text{env } x := \begin{cases} \text{sta } x & \text{if } x \in \underline{A} \\ \text{dyn } x & \text{if } x \in X \end{cases}$$

Using the same reasoning as before and the universal property of the free extension, we can prove partially static equalities on arbitrary models.

2.5 The distributive combination of theories

Given two signatures Σ_1 and Σ_2 , we define their coproduct $\Sigma_1 + \Sigma_2$ as the signature with operators the disjoint union of the operators, but retaining their arity:

$$\Sigma_1 + \Sigma_2 = \{\iota_i f : n \mid (f : n) \in \Sigma_i, i = 1, 2\}$$

Given a Σ_i -term-in-context $n \vdash_{\Sigma_i} t$, we denote by $n \vdash_{\Sigma_1 + \Sigma_2} \iota_i[t]$ the $\Sigma_1 + \Sigma_2$ -term-in-context obtained by applying the injection ι_i to all the operators of t . Similarly, given a Σ_i -equation in context $eq := (n \vdash_{\Sigma_i} t = t')$, we denote by $\iota_i[eq]$ the $\Sigma_1 + \Sigma_2$ -equation in context $n \vdash_{\Sigma_1 + \Sigma_2} \iota_i[t] = \iota_i[t']$.

Let $\mathcal{A}$ and $\mathcal{M}$ be two theories. We define the *distributive combination of $\mathcal{M}$ over $\mathcal{A}$* [Hyland & Power, 2006] to be the theory $\mathcal{M} \triangleright \mathcal{A}$ given by:

$$\begin{aligned} \Sigma_{\mathcal{M} \triangleright \mathcal{A}} &:= \Sigma_{\mathcal{M}} + \Sigma_{\mathcal{A}} \\ \text{Ax}_{\mathcal{M} \triangleright \mathcal{A}} &:= \iota_1[\text{Ax}_{\mathcal{M}}] \cup \iota_2[\text{Ax}_{\mathcal{A}}] \\ &\cup \left\{ \left\langle x_i \right\rangle_{\substack{i=1 \\ i \neq i_0}}^n, \mathbf{y} \mid f(x_1, \dots, x_{i_0-1}, s\mathbf{y}, x_{i_0+1}, \dots, x_n) \right. \left. \begin{array}{l} f : n \in \Sigma_{\mathcal{M}} \\ s : |\mathbf{y}| \in \Sigma_{\mathcal{A}} \end{array} \right\} \\ &\quad = s \langle f(x_1, \dots, x_{i_0-1}, \mathbf{y}_j, x_{i_0+1}, \dots, x_n) \rangle_{j=1}^{|\mathbf{y}|} \end{aligned}$$

Concretely, the signature is the coproduct signature and the axioms are the union of the axioms to which we added distributivity axioms between the operators of $\mathcal{M}$ and $\mathcal{A}$, stating that every operator of $\mathcal{M}$ distributes over every operator of $\mathcal{A}$. We will call $\mathcal{M}$ the multiplicative theory, and $\mathcal{A}$ the additive theory. For example, the theory of semirings is the distributive combination of commutative monoids over monoids. The distributivity axioms state that:

$$\begin{aligned} x, y, z \vdash x \cdot (y + z) &= (x \cdot y) + (x \cdot z) & x, y, z \vdash (x + y) \cdot z &= (x \cdot z) + (y \cdot z) \\ x \vdash x \cdot 0 &= 0 & x \vdash 0 \cdot x &= x \end{aligned}$$

2.6 Monads and distributive laws

Monads are closely related to algebraic theories, and the construction of the distributive combinations in §3 uses the notions of monads and distributive laws from a category-theoretical point of view. We will assume basic knowledge about category theory, mainly about functors, natural transformations and adjunctions. One can refer itself to MacLane [1971].

A *monad* on a category $\mathcal{C}$ is a triple (T, η, μ) where $T : \mathcal{C} \rightarrow \mathcal{C}$ is a functor, $\eta : \text{Id}_{\mathcal{C}} \rightarrow T$ is a natural transformation called the unit, $\mu : T^2 \rightarrow T$ is a natural transformation called the multiplication, and such that the following diagrams commute:

$$\begin{array}{ccc} T & \xrightarrow{T\eta} & T^2 \\ \eta T \downarrow & \searrow & \downarrow \mu \\ T^2 & \xrightarrow{\mu} & T \end{array} \quad \begin{array}{ccc} T^3 & \xrightarrow{T\mu} & T^2 \\ \mu T \downarrow & & \downarrow \mu \\ T^2 & \xrightarrow{\mu} & T \end{array}$$

Every adjunction $\langle F, G, \eta, \varepsilon \rangle$ gives rise to a monad [MacLane, 1971], by defining $T := GF$, $\eta := \eta$ and $\mu := G\varepsilon F$.

Theorem 3 (Zwart and Marsden, 2018)

For a theory $\mathcal{T}$, there is a left adjoint $F^{\mathcal{T}}$ to the obvious forgetful functor $U^{\mathcal{T}} : \mathcal{T}\text{-}\mathbf{Alg} \rightarrow \mathbf{Set}$ where $\mathcal{T}\text{-}\mathbf{Alg}$ is the category with $\mathcal{T}$ -models as objects and $\mathcal{T}$ -model homomorphisms as morphisms. The functor is defined as follows:

- For a set X , $F^{\mathcal{T}}X := \underline{F_{\mathcal{T}}X}$ is the carrier of the free $\mathcal{T}$ -algebra over X .
- The action on a function $h : X \rightarrow Y$ is defined inductively by:

$$\begin{aligned} F^{\mathcal{T}}(h)(x) &= h(x) \quad \text{for } x \in X \\ F^{\mathcal{T}}(h)(\text{op}(t_1, \dots, t_n)) &= \text{op}(F^{\mathcal{T}}(h)(t_1), \dots, F^{\mathcal{T}}(h)(t_n)) \quad \text{for } \text{op} : n \in \Sigma_{\mathcal{T}} \end{aligned}$$

For a theory $\mathcal{T}$, the *free model monad* induced by $\mathcal{T}$ is then the monad induced by the free/forgetful adjunction $U^{\mathcal{T}} \circ F^{\mathcal{T}}$. We say that a monad T corresponds to a theory $\mathcal{T}$ if T is a free model monad induced by $\mathcal{T}$. It is well known that every finitary monad on the category of sets arises as a free model monad for some algebraic theory.

Concretely, the monad T corresponding to the algebraic theory $\mathcal{T}$ is the monad whose terms TX are terms over the variables in X modulo the equations, whose unit $\eta_X : X \rightarrow TX$ sends each variable to the corresponding atomic term and whose multiplication $\mu_X : TTX \rightarrow TX$ is substitution/flattening, i.e., the act of plugging terms into terms. For the theory of (regular) monoids, the corresponding monad is the List monad whose elements LX are finite lists over X , $\eta_X(x) = [x]$ and μ_X is list concatenation.

Given two monads (T, η^T, μ^T) and (S, η^S, μ^S) on a category $\mathcal{C}$, a *distributive law* [Beck, 1969] of T over S is a natural transformation $\lambda : TS \rightarrow ST$ such that the following diagrams commute:

$$\begin{array}{ccc} & T & \\ T\eta^S \swarrow & & \searrow \eta^{ST} \\ TS & \xrightarrow{\lambda} & ST \end{array} \quad \begin{array}{ccc} & S & \\ \eta^{TS} \swarrow & & \searrow S\eta^T \\ TS & \xrightarrow{\lambda} & ST \end{array}$$

$$\begin{array}{ccc} T^2S & \xrightarrow{T\lambda} TST & \xrightarrow{\lambda T} ST^2 \\ \mu^TS \downarrow & & \downarrow S\mu^T \\ TS & \xrightarrow{\lambda} & ST \end{array} \quad \begin{array}{ccc} TS^2 & \xrightarrow{\lambda S} STS & \xrightarrow{S\lambda} S^2T \\ T\mu^S \downarrow & & \downarrow \mu^{ST} \\ TS & \xrightarrow{\lambda} & ST \end{array}$$

These equations ensure that the distributive law is compatible for both the units and the multiplications of S and T . This law induces a composite monad ST on $\mathcal{C}$ with unit $\eta^{ST} = \eta^S \eta^T$ and multiplication $STST \xrightarrow{S\lambda T} SSTT \xrightarrow{\mu^S \mu^T} ST$.

give example/intuition about what a distributive law does (example of the list monad over the Abelian group monad), and how this gives the product of the composite monad

3 Categorical approach to the construction of distributive combinations

While using FREX does not require knowledge about category theory, FREX uses many category-theoretic concepts internally. Therefore, the construction of distributive combinations is firstly category-theoretic. However we won't go into details here, as this would require a course on 2-categories and multi-categories.

3.1 Challenge about constructing distributive combinations

Constructing generic free algebras and free extensions for arbitrary theories is not hard. In fact, §2.2 gives an explicit construction for the free algebra of any theory. However, the equivalence relation between terms in the quotient is not necessarily effective/decidable. This is problematic since we've seen in §2.4 that we need to be able to prove equalities in the free algebra to prove any equation with free algebras.

Free extensions also have a generic construction: Allais et al. [2025] and Yallop et al. [2018] state that the free extension of an algebra A by X is the category-theoretic coproduct of A with the free $\mathcal{A}$ -algebra over X . However, there is no generic formula for the coproduct of two algebras for an arbitrary theory.

The difficulty of the construction is for it to be effective so that equivalence between two terms can be determined computationally. This forbids the use of quotients, category theory pushouts etc...

3.2 Initial result about the free algebra for the distributive combination

A first result was given by Ohad Kammar (my internship supervisor) about the free algebra for the distributive combination of a theory over a commutative theory. A theory $\mathcal{A}$ is commutative iff for every $f : n \in \Sigma_{\mathcal{A}}$ and every $g : m \in \Sigma_{\mathcal{A}}$,

$$f \left(\begin{array}{c} g(x_{11}, \dots, x_{1m}) \\ \vdots \\ g(x_{n1}, \dots, x_{nm}) \end{array} \right) = g \left(f \left(\begin{array}{c} x_{11} \\ \vdots \\ x_{n1} \end{array} \right) \cdots f \left(\begin{array}{c} x_{1m} \\ \vdots \\ x_{nm} \end{array} \right) \right) \quad (1)$$

For monoids, this states that

$$(x \cdot y) \cdot (z \cdot w) = (x \cdot z) \cdot (y \cdot w) \quad 0 \cdot 0 = 0 \quad 0 = 0$$

The theory of commutative monoids is therefore a commutative theory, fortunately. Note that every operator of arity 0 and 1 commutes with itself.

Theorem 4 (Free algebra for the distributive combination, incomplete)

Let $\mathcal{A}$ be a commutative theory, and $\mathcal{M}$ be any theory. Then there exists a distributive law of the free model monad induced by $\mathcal{M}$ over the free model monad induced by $\mathcal{A}$, and the free model monad induced by $\mathcal{M} \triangleright \mathcal{A}$ is the composite monad of the two free model monads.

Intuitively, this means that given two free algebras for the additive and multiplicative part, composing them gives the free algebra for the distributive combination of the theories. This construction is effective in the sense of §3.1.

Even though the construction itself was well-defined, the proof wasn't complete and that for a good reason: the statement is incorrect. The culprit is the absence of distributive laws under some hypothesis.

3.3 Failure of the first result

Zwart and Marsden [2018] gives multiple results about the absence of distributive laws under some hypothesis. We can regroup these results under 3 categories:

- ▷ theorems 3.5, 3.12 and 4.24 all forbid the existence of an idempotent operator in the multiplicative theory, on top of other conditions.

- ▷ theorem 4.7 forbids the existence of two distinct constants in the additive theory.
- ▷ theorem 4.17 imposes the abides equation $(a * b) * (c * d) = (a * c) * (b * d)$ to hold for every operator $*$ in the additive theory.

The second and third point are both taken care of by the hypothesis of commutativity of the additive theory in Theorem 4. Two constants commute with each other if they are equal, and the abides equation is the commutativity hypothesis for $f = g = *$.

However the first point is not accounted for. Under the current hypothesis, idempotent operators are allowed. Therefore, we need to strengthen our hypothesis.

3.4 Corrected result

We strengthen the hypothesis on $\mathcal{M}$ by requiring it to be an affine theory. A theory $\mathcal{M}$ is *affine* if every axiom is an equation between terms where each variable occurs at most once. This allows weakening/deletion of variables, but not contraction. For example, $x, y \vdash x + y = y + x$ or $x \vdash x \cdot 0 = 0$ are affine equations, but $x \vdash x \vee x = x$ is not. Semigroups, monoids, groups and their commutative variants are all affine theories, whereas semilattices are not. This effectively covers the first point of the previous section.

We now state the corrected theorem.

Theorem 5 (Free algebra for the distributive combination)

Let $\mathcal{A}$ be a commutative theory, and $\mathcal{M}$ be an affine theory. Then there exists a distributive law of the free model monad induced by $\mathcal{M}$ over the free model monad induced by $\mathcal{A}$, and the free model monad induced by $\mathcal{M} \triangleright \mathcal{A}$ is the composite monad of the two free model monads.

The actual construction uses operads, translations from operads to theories, 2-categories and multicategories. Therefore, we won't present it nor the proof as is. We are gonna explain the gist of it.

First, start by noticing that a model for the distributive combination is made of a $\mathcal{M}$ -model and a $\mathcal{A}$ -model on the same carrier set (this ensures that the respective axioms hold) where moreover the operations of $\mathcal{M}$ distribute over the operations of $\mathcal{A}$ (this is a pullback in the category of categories). Let X be a set of variables. The construction start by taking the free $\mathcal{M}$ -algebra over X , $A_0 := F_{\mathcal{M}}X$. We then take the free $\mathcal{A}$ -algebra over $F_{\mathcal{M}}X$, $A_1 := F_{\mathcal{A}}(F_{\mathcal{M}}X)$. This gives us our candidate $\mathcal{A}$ -model. This step corresponds to the monadic composition, minus the multiplications. But we now need to define the semantics of the operators of $\mathcal{M}$ on the carrier A_1 . To do this, we "lift" the $\mathcal{M}$ -operators so that the lifted operators on the new carrier distribute over the $\mathcal{A}$ -operators. This is what gives the distributive law. Through some complex reasoning and with the help of the affine hypothesis, we prove that the $\mathcal{M}$ -axioms hold. The commutativity hypothesis of $\mathcal{A}$ is used to prove the existence of an adjunction between multicategories, which is used to lift the $\mathcal{M}$ -operators.

This construction is effective: it is solely built from applying the free algebra constructions and some 2-functors in well-chosen categories. The lifted operators can be defined concretely.

3.5 An attempt at constructing the free extension for the distributive combination

The natural next step is wanting to extend this result to free extensions. However the task is not so easy.

Not easy does not mean impossible. We managed to define a construction that uses pushouts and the free extensions of the component theories. While not effective because of the quotient nature of pushouts, this proves that it is indeed possible to use the free extensions of the composite theories to construct the one for the combined theory.

The construction of free extensions for distributive combinations is still ongoing work. We need to find a new pullback that is suited for extensions, and we need to find a new adjunction between the multi-category of extensions and the obvious forgetful functor.

4 Implementation

FREX has multiple implementations, first in Haskell and MetaOCaml [Yallop et al., 2018], then in Agda [Corbyn, 2021] and finally in Idris 2 [Allais et al., 2025]. We are interested in the implementation in Idris 2, a purely functional programming language with first class types. We won't present any code here as it doesn't add anything to the comprehension of the project.

4.1 Setoids

FREX uses *setoids* [Bishop, 1967; Hofmann, 1997] instead of simple sets. This offers them some additional functionalities on top of term simplification and equation solving: printing terms and proofs, proof simplification, code generation and more [Allais et al., 2025].

A *setoid* is a set X equipped with an equivalence relation $\sim_X$. A *setoid homomorphism* $f : X \rightarrow Y$ is a function compatible with the equivalence relation: if $x \sim_X y$ then $f(x) \sim_Y f(y)$. Every set X can be seen as a setoid by equipping it with the equality relation ($=$). However setoids lets us define some more complicated equivalence relations. For example, we can consider the setoids of terms for some theory $\mathcal{T}$ and set of variables X , equipped with the provability relation from Fig 1. Two equivalent terms in this setoid represent different, but provably equal, terms. This is different from the quotient $(\text{Term}_{\Sigma_{\mathcal{T}}} X) / (X \vdash_{\mathcal{T}})$ where elements represent equivalence classes of provably equal terms.

FREX implements a slightly different version of the universal algebra presented in §2. Carriers for algebras are setoids instead of sets. Therefore, all homomorphisms must also be setoid homomorphisms.

4.2 Implementing the distributive combination

Not the entire distributive combination of free algebras has been implemented in this internship. Only the semantics have been implemented, it remains to prove in Idris 2 that the constructed algebra validates the axioms and that it is indeed free. However, since we do have the full theoretical proof, this is just a matter of taking the time and writing the remaining proofs. This can take a long time as the formalism tends to be quite heavy.

We were however still able to use the current implementation to prove some equations on specific examples. For instance, for every example of a distributive combination we implemented (see the Table 1), we proved that all of the axioms of the combination hold

using the reflexivity of the equivalence relation of the carrier. This also lets us test the performance of our implementation in §4.3.

4.3 Performance evaluation

Following the work of Allais et al., 2025, we tested our implementation to find the instantaneity (0.1s), interactivity (1s) and attention (10s) thresholds described by Nielsen, 1994. This is important as developping in Idris2 is interactive and our tool is meant to discharge the user of painful arithmetic proofs.

FREX’s checking time¹ remains under the instantaneity threshold for terms of size 30 to 60 depending on the number of free variables, as pictured in Figure 2. This is more than sufficient to deal with the typical equalities that arise in dependently typed programs.

Add figure

Figure 2: caption

Comparing the execution times for semirings and rings with the ones in Allais et al. [2025] for monoids, the performance seems to have much improved. This can be explained by two different factors:

- Allais et al. [2025] mentioned a performance bottleneck in Idris2’s evaluator, this was four years ago and the developpment of Idris2 probably eliminated this bottleneck.
- The tests have not been run on the same machine, and the CPU used for the semirings test is much more efficient than the one used for the monoids tests.

5 Comparison with existing solvers

In dependently-typed programming languages, the state-of-the-art for polynomial equalities over commutative semirings/rings was originally presented by Grégoire and Mahboubi [2005]. This is the current implementation of Rocq’s `ring` tactic, and it was also adopted by Adga’s ring solver [Kidney, 2019]. We’ll explain briefly how the two approaches differ, and then compare the capabilities of both algorithms.

5.1 Reflection

Reflection is using the ability of the Calculus of Constructions to speak and reason about itself. Here is the pipeline for the reflection process used in Rocq described by Grégoire and Mahboubi [2005].

Consider a ring structure A , with two constants 0 and 1, three binary operators $+$, $*$ and $-$ and an opposite unary function $-$. We want to prove the equality of two terms t_1 and t_2 modulo the ring axioms. To do so, they introduce two inductive types `PolExpr` and `Pol`, two evaluation functions $\varphi_P : \text{PolExpr} \rightarrow A$ and $\varphi_{PE} : \text{Pol} \rightarrow A$ and a translation function $\mathfrak{T} : A \rightarrow \text{PolExpr}$. `PolExpr` describes the syntax of terms of type A , while `Pol` describes normal forms of terms of type A . The reflection process is pictured in Figure 3. It goes as follows:

▷ from t_1 and t_2 , build two corresponding terms e_1 and e_2 with $\mathfrak{T}$.

¹We used an Intel Core Ultra 7 255HX machine with 16GB memory, running Idris 2 version 0.8.0-6a54860ee on Debian Linux 13.

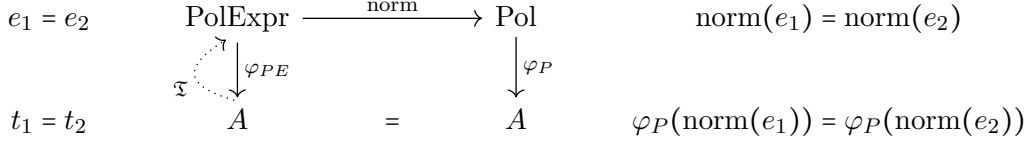


Figure 3: Reflection process

▷ normalise these two polynomials using `norm`

▷ prove the equality $\text{norm}(e_1) = \text{norm}(e_2)$ to conclude for the equality t_1 and t_2 .

In order for this process to be correct, there are some constraints on φ_P, φ_{PE} and $\mathfrak{T}$ that need to be met:

$$\forall e \in \text{PolExpr}, \varphi_{PE}(e) = \varphi_P(\text{norm}(e)) \quad (2)$$

$$\forall a \in A, \varphi_{PE}(\mathfrak{T}(a)) = a \quad (3)$$

If the equality of the normal forms hold, it follows that:

$$t_1 = \varphi_{PE}(\mathfrak{T}(t_1)) = \varphi_P(\text{norm}(\mathfrak{T}(t_1))) = \varphi_P(\text{norm}(\mathfrak{T}(t_2))) = \varphi_{PE}(\mathfrak{T}(t_2)) = t_2$$

The type `Pol` is built so that equality can be checked syntactically: if two polynomials $P_1, P_2 \in \text{Pol}$ are equal then so are their syntax tree. Therefore, equality of normal form comes for free and one only needs to prove the correctness criterion (2) for the tactic to be correct.

5.2 Almost-rings

The `ring` tactic is capable of dealing both with commutative rings and commutative semirings, under the same implementation. To unify both structure, they resort to *almost-rings*, an intermediate structure similar to rings and semirings.

An *almost-ring* is a commutative semiring completed with an unary operator called *almost-opposite*. The almost-opposite, denoted by $-$, satisfies the following axioms:

$$x, y \vdash -(x * y) = -x * y$$

$$x, y \vdash -(x + y) = -x + -y$$

$$x, y \vdash x - y = x + -y$$

The last equation defines an associated *pseudo-subtraction*.

These axioms do not allow to prove the missing axiom defining the opposite in a ring $x \vdash x + -x = 0$. This lets them equip a commutative semi-ring with an almost-ring structure by instantiating the almost-opposite by the identity function, which satisfies the axioms. Therefore, implementing a solver for almost-rings is sufficient to deal with both commutative semirings and commutative rings.

Even though the equation $x \vdash x + -x = 0$ is not provable in an almost-ring, it will be proved for rings by the `ring` tactic as long as $1 + (-1)$ reduces to 0 in the coefficient set used in the types `Pol` and `PolExpr`.

5.3 Soundness and completeness

Both the FREX solvers and the `ring` tactic are sound and complete, but with a subtlety.

Any `fral`/`frex` is canonically isomorphic to the quotient `fral`/`frex`, and an effective `fral`/`frex` with a constructive universality proof has an effective isomorphism to the quotient setoid. In this way, a simplifier using an effective `fral`/`frex` is constructively sound and complete [Allais et al., 2025]. This completeness is with respect to the underlying theory of the `fral`/`frex`, not with respect to all actually provable equalities in the model for which we are trying to prove an equality. For instance, considering the ring of booleans with logical or as addition and logical and as multiplication, the addition is idempotent: $\forall x : \text{Bool}, x \vee x = x$. Even though it holds, the equation is not a consequence of the ring axioms and therefore it is unable to be solved by a `fral` for rings. Since both addition and multiplication are idempotent for this ring, we are not under the hypothesis of Theorem 5 and therefore FREX cannot be used to solve every equation over the booleans.

The soundness of the `ring` tactic comes from the correctness lemma (2). `ring` computes the normal forms for the LHS and the RHS of the equation and checks whether they are equal or not. If they are equal, (2) assures us that the two terms are indeed equal, thus giving us soundness. Completeness with respect to the commutative semiring/ring theory is ensured by the validity of the normal form used (sparse Horner forms) and the equation (3). The validity of the normal forms ensure that if two polynomial expressions represent the same polynomial, then their normal form will be equal. Equation (3) ensures that the translation process from expressions to normal forms is correct. However, the equations provable by the `ring` tactic actually depend on which coefficient set is used in the types `Pol` and `PolExpr`. By default, the `ring` tactic takes $\mathbb{Z}$ as its coefficient set, but it may not always be the best choice. For instance, for solving equations over the ring of booleans, taking the booleans themselves can let `ring` solve equations such as $\forall x : \text{Bool}, x \vee x = x$. This is because $1 + 1 = 1$ for booleans. The left side of the equality is reflected to $X + X$, whose normal form $(1 + 1) \cdot X$ is reduced to $1 \cdot X = X$.

Therefore, both FREX and the `ring` tactic are sound and complete with respect to the underlying algebraic theories, but a good choice for the coefficient set of the `ring` tactic can lead to solving more equations.

5.4 Comparison

5.4.1 Supported theories

The FREX approach to algebraic simplification lets us tackle many more theories than the Rocq `ring` tactic can. In Table 1, we show examples of models for the different distributive combinations of mere/commutative mere/involutive semigroups, monoids and groups. In red are the theories that the `ring` tactic can handle.

The models in the table are the following:

- $\mathbb{N}, \mathbb{Z}, \mathbb{R}$: natural numbers, integers and real numbers with the usual operations.
- $(\mathbb{C}, \overline{})$: complex numbers with conjugation.
- $\mathbb{C}_{>0} := \{a + bi \mid a \in \mathbb{R}_{>0}, b \in \mathbb{R}\}$ are complex numbers with a strictly positive real part.
- $2\mathbb{Z}[i] := \{2(a + bi) \mid a, b \in \mathbb{Z}\}$ with standard addition, multiplication and conjugation as involution.
- $\mathcal{P}(\Sigma^*)$ and variants: formal languages on the alphabet Σ , with union as addition and concatenation as multiplication.

Multiplicative structure			Additive structure		
Theory	Comm.	Inv.	CSemigroup	CMonoid	AbGroup
Semigroup	Mere	Mere	$\mathcal{P}(\Sigma^+) \setminus \{\emptyset\}$	$\mathcal{P}(\Sigma^+)$	$\mathbb{Z}\langle \Sigma^+ \rangle$
		Inv.	$(\mathcal{P}(\Sigma^+) \setminus \{\emptyset\}, -^R)$	$(\mathcal{P}(\Sigma^+), -^R)$	$(\mathbb{Z}\langle \Sigma^+ \rangle, \text{rev})$
	Comm.	Mere	$2\mathbb{N}_{>0}$	$2\mathbb{N}$	$2\mathbb{Z}$
		Inv.	$\mathfrak{m}_n^\sigma \setminus \{0\}$	$\mathfrak{m}_n^\sigma$	$(2\mathbb{Z}[\mathbf{i}], -)$
Monoid	Mere	Mere	$\mathcal{P}(\Sigma^*) \setminus \{\emptyset\},$ $M_n(\mathbb{N}_{>0})$	$\mathcal{P}(\Sigma^*), M_n(\mathbb{N})$	$M_n(\mathbb{Z})$
		Inv.	$(M_n(\mathbb{N}_{>0}), -^T),$ $(\mathcal{P}(\Sigma^*) \setminus \{\emptyset\}, -^R)$	$(M_n(\mathbb{N}), -^T),$ $(\mathcal{P}(\Sigma^*), -^R),$ $\text{Rel}(X)$	$(M_n(\mathbb{Z}), -^T)$
	Comm.	Mere	$\mathbb{N}_{>0}$	$\mathbb{N}$	$\mathbb{Z}, \mathbb{R}, \mathbb{C}$
		Inv.	$\mathbb{N}_\sigma \llbracket X_1, \dots, X_n \rrbracket \setminus \{0\}$	$\mathbb{N}_\sigma \llbracket X_1, \dots, X_n \rrbracket$	$(\mathbb{C}, -),$ $\mathbb{Z}_\sigma \llbracket X_1, \dots, X_n \rrbracket$
Group	Mere	Mere	$T_{n, \Delta_{>0}}(\mathbb{R})$	impossible nontrivially	impossible nontrivially
		Inv.	$(T_{n, \Delta_{>0}}(\mathbb{R}), -^\dagger)$	impossible nontrivially	impossible nontrivially
	Comm.	Mere	$\mathbb{R}_{>0}$	impossible nontrivially	impossible nontrivially
		Inv.	$(\mathbb{C}_{>0}, -)$	impossible nontrivially	impossible nontrivially

Table 1: Examples for the distributive combination of theories

- $M_n(X)$: $n \times n$ square matrices with coefficients in X . $-^T$ denotes matrix transposition.
- $\mathbb{Z}\langle \Sigma^+ \rangle$: finite integer linear combinations of nonempty words, i.e, finitely supported functions from Σ^+ to $\mathbb{Z}$. Addition is pointwise function addition and multiplication is concatenation that distributes over sums. rev is word reversal.
- $\text{Rel}(X)$: the set of binary relations on X . Addition is union, and multiplication is composition. Involution is the converse $R^\dagger := \{(y, x) \mid (x, y) \in R\}$.
- $Y_\sigma \llbracket X_1, \dots, X_n \rrbracket$: multivariate generating functions in the variables $X_1, \dots, X_n$ with coefficients in Y . Addition and multiplication are the standard operations, the involution is defined by $f(x_1, \dots, x_n)^{\ast_\sigma} := f(x_{\sigma(1)}, \dots, x_{\sigma(n)})$ where $\sigma \in \mathfrak{S}_n$ is a permutation of order 2, i.e, $\sigma^2 = \text{id}$.
- $\mathfrak{m}_n^\sigma := \{f \in \mathbb{N}_\sigma \llbracket X_1, \dots, X_n \rrbracket \mid f(0, \dots, 0) = 0\}$: multivariate generating functions in the variables $X_1, \dots, X_n$ with coefficients in $\mathbb{N}$ and with no constant part.
- $(T_{n, \Delta_{>0}}(\mathbb{R}), -^\dagger)$: upper triangular matrices with coefficients > 0 on the diagonal. Addition and multiplication is standard matrix addition and multiplication. The involution $-^\dagger$ reverses the order of the coefficients on the diagonal, e.g, $\Delta(1, \dots, n)^\dagger = \Delta(n, \dots, 1)$.

Some combinations are impossible nontrivially. This is the case of every combination of a multiplicative group with an additive structure that has a 0 element. In such a case, every element are equal to each other. Indeed, suppose that 0 is inversible, i.e, there exists 0^{-1} such that $0 \cdot 0^{-1} = 1$. Because of the distributivity axioms, $x \vdash x \cdot 0 = 0$ holds in all of these cases. We also have the associativity of the multiplication, therefore for every element a , $a = a \cdot (0 \cdot 0^{-1}) = (a \cdot 0) \cdot 0^{-1} = 0 \cdot 0^{-1} = 1$ and every element is equal to 1. The only model validating these theories is the trivial model $\{0\}$.

5.4.2 Additional equalities

The Rocq `ring` tactic supports² taking additional equalities as arguments to solve equalities between terms. This lets one prove goals such as

```
Goal forall a b : Z, 2 * a * b = 30 -> (a + b) ^ 2 = a ^ 2 + b ^ 2 + 30.
Proof.
```

```
  intros a b H; ring [H].
```

```
Qed.
```

This is impossible with the FREX approach. The universal property of free algebras and free extensions guarantees that the only equations that are valid in the free algebra/extension are the ones that are provable with the provability relation of Figure 1. Due to the nature of the solving algorithm (§2.4), one can't provide additional facts to the solver, and therefore the goal above is not provable with FREX.

However this feature is limited. The supported additional equalities need to be of the form $m = p$ where m is a monomial and p is a polynomial in the corresponding ring structure. Therefore, you can't reason about any other operators than $+$ and $*$, even when giving additional axioms concerning the operator. For instance, define an abstract ring and add an involution operator $\sim$. Even by giving the axioms

```
Rinv_inv : forall x, ~ (~ x) == x.
Rinv_antidistr : forall x y, ~ (x * y) == ~ y * ~ x.
```

to the `ring` tactic, it is unable to solve the basic goal

```
Lemma invInv : forall x, ~ (~ x) == x.
```

```
Proof.
```

```
  intros.
```

```
  Fail ring [(Rinv_inv x) (Rinv_antidistr x)].
```

```
Abort.
```

because $\sim$ is not a part of the ring structure. This is to be expected when looking at how reflection work in §5.1. Because of this, some theories such as involutive semirings or involutive rings which are supported by FREX can't be supported by the `ring` tactic.

5.4.3 Other Frex capabilities

We presented FREX as a generic algebraic simplification framework, but it is not limited to that. All of the following capabilities of FREX cannot be achieved by the `ring` tactic.

By appealing to the universal property of the `fral/frex`, every `fral/frex` is isomorphic to the quotient `fral/frex`. This lets us extract a proof certificate, i.e, a derivation for the equation for the provability relation of Figure 1. FREX can package such a derivation as

²See <http://rocq-prover.org/doc/V9.2.0/refman/addendum/ring.html>.

a lemma that is valid for any model of the given theory. Users can then seamlessly invoke these lemmas in any model, avoiding further FREX calls [Allais et al., 2025, Section 5.2].

The obtained derivation can also be used for pretty-printing human-readable proofs, going through each step and specifying which axiom is used and where each time. It can also be used to print Idris proofs for the lemmas [Allais et al., 2025, Appendix C].

add example

FREX can also be used to normalise code. Corbyn et al. [2022] present a normalisation by evaluation framework using FREX that can simplify higher-order functional programs with algebraic structure. For instance, the framework can simplify simply typed lambda calculus terms extended with an arbitrary commutative monoid M . Take M to be the commutative monoid of integers with multiplication and FREX can normalise terms such as $\lambda x.(2 * 3) * x$ or $\lambda x.(\lambda y.2 * y)(x * 3)$ to $\lambda x.6 * x$, combining beta-reduction and algebraic simplification. While a naive normaliser suffices to reduce the first term to its normal, reducing the second term requires reordering the terms and invoking the axioms of the commutative monoid, which requires FREX.

The FREX simplification can also be applied to multi-stage programming. In Yallop et al. [2018, Section 4.3], FREX is used to simplify the code generated by a staged version of `sprintf` to produce more efficient code requiring less string concatenations, reducing expressions such as `(((" ++ show x) ++ "a") ++ "b") ++ show y to show x ++ ("ab" ++ show y)`.

5.4.4 Runtime comparison

comparison in usability (runtime) $\rightarrow$ frex might be slower as typechecking takes a long time.

TODO: runtime comparison

is this really relevant?

6 Conclusion

6.1 Related work

read and cite Fujii et al., 2026 + a future direction is to systematically compare with their work

6.2 Future work

construction for free extensions

multi-sorted version (Allais et al., 2023) for distributive combination and just formalisation in general

partial fields

References

- Allais, G., Brady, E., Corbyn, N., Kammar, O., & Yallop, J. [2025]. Frex: Dependently typed algebraic simplification. *Proceedings of the ACM on Programming Languages*, 9[ICFP], 30–65. <https://doi.org/10.1145/3747506>
- Allais, G., Corbyn, N., Lindley, S., & Yallop, J. [2023]. Modular construction of multi-sorted free extensions (short paper). <https://api.semanticscholar.org/CorpusID:260005086>

- Beck, J. [1969]. Distributive laws. In B. Eckmann [Ed.], *Seminar on triples and categorical homology theory* [pp. 119–140]. Springer Berlin Heidelberg.
- Bishop, E. [1967]. *Foundations of constructive analysis*. McGraw-Hill.
- Burris, S., & Sankappanavar, H. [1981, January]. *A course in universal algebra* [Vol. 91]. <https://doi.org/10.2307/2322184>
- Corbyn, N. [2021]. *Proof synthesis with free extensions in intensional type theory* [tech. rep.] [MEng Dissertation]. University of Cambridge.
- Corbyn, N., Kammar, O., Lindley, S., Valliappan, N., & Yallop, J. [2022]. Normalization by evaluation with free extensions.
- Fujii, S., Tsai, Y. C., Montacute, Y., & Hasuo, I. [2026]. Monads and Distributive Laws in Substructural Contexts. In C. Faggian & J.-P. Katoen [Eds.], *41st annual symposium on logic in computer science (lics 2026)* [45:1–45:28, Vol. 380]. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. <https://doi.org/10.4230/LIPIcs.LICS.2026.45>
- Keywords: Monad, distributive law, operad, category theory, effect.
- Grégoire, B., & Mahboubi, A. [2005]. Proving equalities in a commutative ring done right in coq. In J. Hurd & T. Melham [Eds.], *Theorem proving in higher order logics* [pp. 98–113]. Springer Berlin Heidelberg.
- Hofmann, M. [1997]. *Extensional constructs in intensional type theory* [C. J. Rijsbergen, Ed.]. Springer-Verlag.
- Hyland, M., & Power, J. [2006]. Discrete lawvere theories and computational effects [Algebra and Coalgebra in Computer Science]. *Theoretical Computer Science*, 366[1], 144–162. <https://doi.org/https://doi.org/10.1016/j.tcs.2006.07.007>
- Kidney, D. O. [2019]. Automatically and efficiently illustrating polynomial equalities in agda. URL: <https://doisinkidney.com/pdfs/bsc-thesis.pdf>.
- MacLane, S. [1971]. *Categories for the working mathematician* [Graduate Texts in Mathematics, Vol. 5]. Springer-Verlag.
- Nielsen, J. [1994]. *Usability engineering*. Morgan Kaufmann Publishers Inc.
- Yallop, J., von Glehn, T., & Kammar, O. [2018]. Partially-static data as free extension of algebras. *Proc. ACM Program. Lang.*, 2[ICFP]. <https://doi.org/10.1145/3236795>
- Zwart, M., & Marsden, D. [2018]. Don’t try this at home: No-go theorems for distributive laws. <https://arxiv.org/abs/1811.06460>

Appendices

A Internship context

B Source code

The source code can be found on a fork of the **Idris Frex** project at <https://github.com/S-c-r-a-t-c-h-y/idris-frex/tree/amaury>.

C Proof of Theorem 5

The full proof of Theorem 5 is more than 50 pages long and is compiled in a single file at <https://github.com/S-c-r-a-t-c-h-y/semiring-solvers/blob/main/proof/semirings.pdf>.

D Research notes

My research notes can be found on my GitHub at <https://github.com/S-c-r-a-t-c-h-y/semiring-solvers/blob/main/notes/notes.pdf>. However, I do not recommend reading them as it is a draft and was not meant to be read by other people.

Acknowledgments